

Jira Task 1 - Monolith vs Microservices Architecture

Objective:

Understand why companies move from Monolithic application to Microservices.

Step 1 — Study the concepts

- Understand Monolithic architecture.
- Understand Microservice architecture.
- Understand Modular monolith.
- Understand Distributed systems.
- Understand Service boundaries.
- Understand Independent deployment.
- Understand Scalability.

1. Monolithic Architecture

A single application where all modules are connected and usually deployed together.

Example: User + Product + Order modules are inside one Spring Boot project.

2. Microservices Architecture

The application is divided into small, independent services.

Example:

* User Service

* Product Service

* Order Service

Each service can be developed and deployed independently.

3. Modular Monolith

Still one application, but the code is organized into separate modules.

4. Distributed Systems

Different services run on different servers or processes and communicate through a network.

5. Service Boundaries

Each microservice should have a clear responsibility. Example: Order Service handles only orders and should not manage user details.

6. Independent Deployment

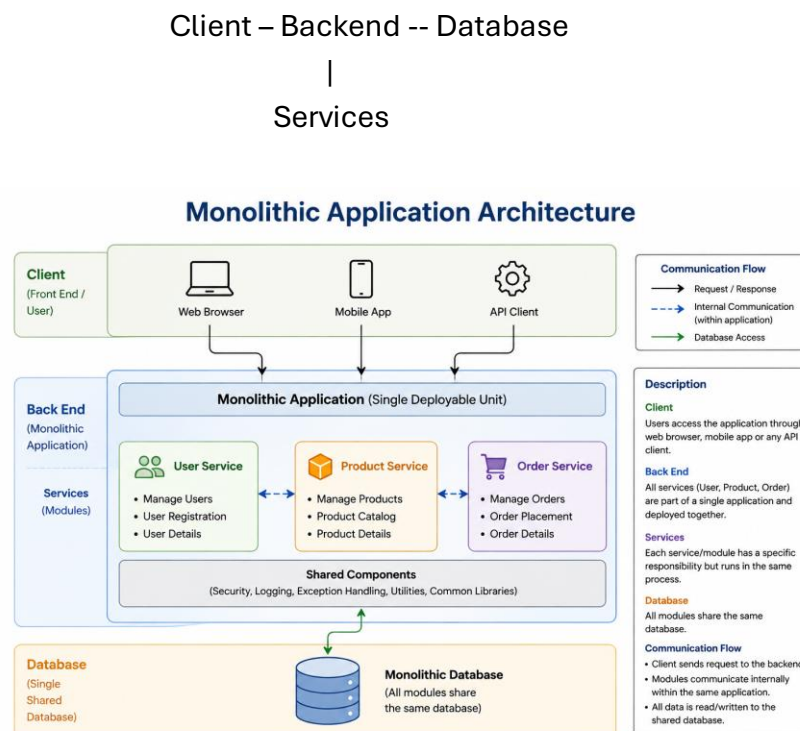
You can deploy one service without deploying the entire application.

7. Scalability

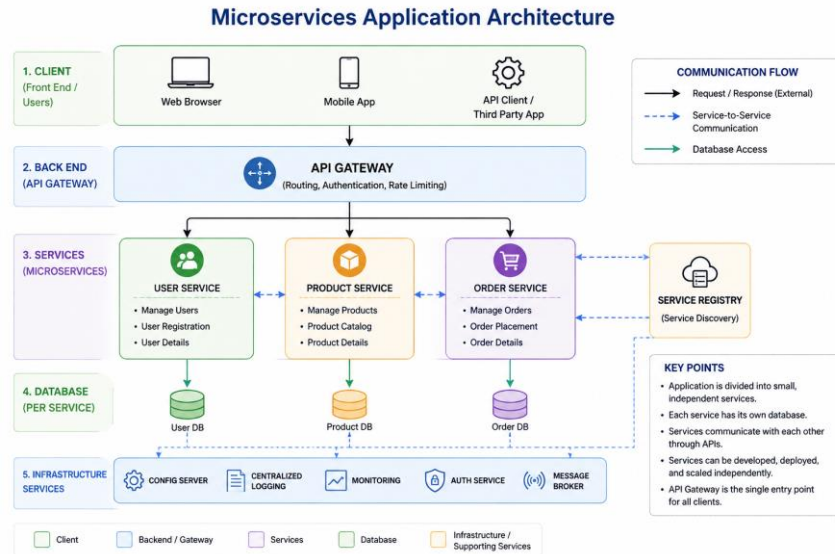
The ability of an application to handle more users or traffic.

Step 2 - Create Architecture Diagrams

Architectural Diagram for Monolithic Application



Architectural Diagram for Microservices application



Step 3 – Create a sample Java application

Project Structure

Monolith Microservices

```
|-----src/main/java/com/monolith
|       |---- MonolithApplication.java
|       |
|       |---- user
|       |       |---- User.java
|       |       |---- UserController.java
|       |
|       |---- product
|       |       |---- Product.java
|       |       |---- ProductController.java
|       |
|       |---- order
|       |       |---- Order.java
|       |       |---- OrderController.java
|
|-----src/main/resources
|       |---- application.Properties
```

1.MAIN SPRINGBOOT APPLICATION

MonolithApplication.java

```
package com.monolith;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MonolithApplication {

    public static void main(String[] args) {
        SpringApplication.run(MonolithApplication.class, args);
    }
}
```

2.USER MODULE

User.java

```
package com.monolith.user;

public class User {

    private Long id;
    private String name;
    private String email;

    public User(Long id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public Long getId() {
        return id;
    }
}
```

```

    public String getName() {
        return name;
    }
    public String getEmail() {
        return email;
    }
}

```

***UserController.java**

```

package com.monolith.user;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    @GetMapping("/users")
    public User getUser() {
        return new User(1L, "Raju", "raju@email.com");
    }
}

```

3. PRODUCT MODULE

Product.java

```

package com.monolith.product;

public class Product {
    private Long id;
    private String name;
    private double price;
}

```

```

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public double getPrice() {
        return price;
    }
}

```

***ProductController.java**

```

package com.monolith.product;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ProductController {

    @GetMapping("/products")
    public Product getProduct() {
        return new Product(1L, "Laptop", 50000);
    }
}

```

3. ORDER MODULE

Order.java

```
package com.monolith.order;

public class Order {

    private Long id;
    private Long userId;
    private Long productId;

    public Order(Long id, Long userId, Long productId) {
        this.id = id;
        this.userId = userId;
        this.productId = productId;
    }

    public Long getId() {
        return id;
    }

    public Long getUserId() {
        return userId;
    }

    public Long getProductId() {
        return productId;
    }
}
```

***OrderController.java**

```
package com.monolith.order;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class OrderController
```

```
@GetMapping("/orders")
public Order getOrder() {
    return new Order(1L, 1L, 1L);
}
}
```

Step 4 — Identify Problems

1. Deployment Problems

In a monolithic application, all modules are deployed together. If we make a small change to the User module, we may need to redeploy the entire application, including the Product and Order modules.

2. Scaling Problems

All modules are scaled together. For example, if only the Order module receives heavy traffic, we still must scale the entire application, which can waste resources.

3. Database Coupling

The User, Product, and Order modules often share one database. Changes to database tables can affect other modules and make the application difficult to modify independently.

4. Code Coupling

Modules inside the monolithic application can become tightly connected. A change in one module may affect other modules, making the code more difficult to maintain.

5. Team Dependency

Different development teams may work on different modules, but because everything is in one application, teams can depend on each other for changes, testing, and deployment.

6. Failure Impact

If one important part of the application fails, it can affect the entire application. For example, a problem in the Order module may cause the whole monolithic application to become unavailable.

Step 5 — Convert the Design

1. User Service

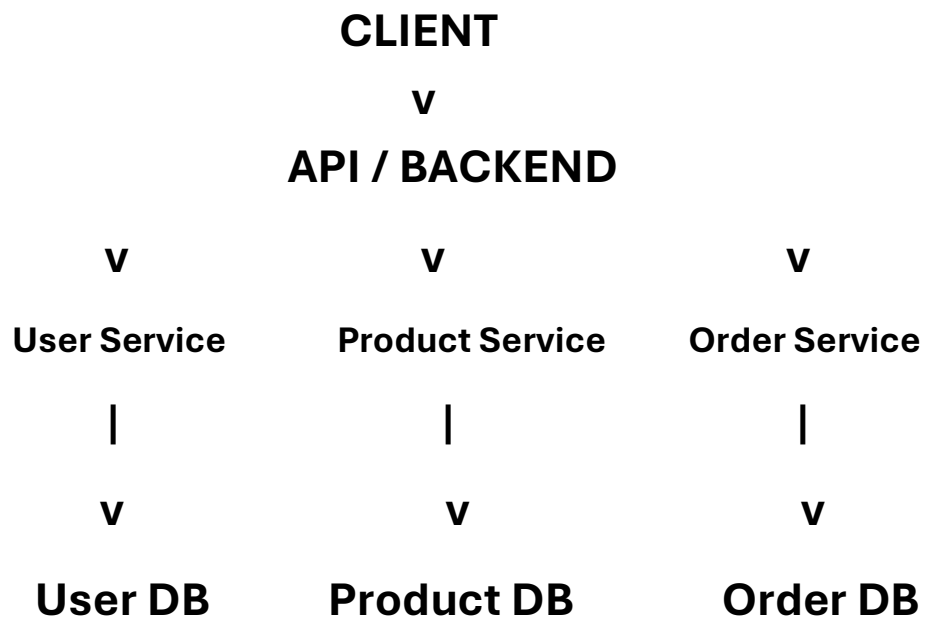
The User Service manages all user-related operations, such as creating users, viewing user details, and updating user information.

2. Product Service

The Product Service manages product-related operations, such as adding products, viewing products, and updating product information.

3. Order Service

The Order Service manages order-related operations, such as creating orders, viewing order details, and processing orders.



Main advantage

Each service can be developed, deployed, scaled, and maintained independently.

This reduces the problems you identified in Step 4.

Step 6 — Compare Both Architectures

In this step, compare the **Monolithic Architecture** and **Microservices Architecture** based on the following points:

Area	Monolithic Architecture	Microservices Architecture
Deployment	All modules are developed in one application. Changes can affect other modules.	Each service, such as User, Product, and Order, can be developed independently.
Deployment	The entire application is deployed together.	Each service can be deployed independently.
Scaling	The whole application must be scaled, even if only one module needs More resources.	Only the service with heavy traffic can be scaled, for example, the Order Service.
Maintenance	Can become difficult as the application grows because all code is connected.	Easier to maintain because Each service has a separate responsibility.
Testing	Testing may take longer because Changes can affect the entire application.	Each service can be tested independently, although service Communication also needs testing.
Failure Handling	A major failure in one module may affect the whole application.	A failure can often be isolated to one service, reducing the impact on other services.

Conclusion

A Monolithic Architecture is usually simpler for small applications, while Microservices Architecture is more suitable for larger and complex applications that need independent deployment, scaling, and maintenance.

Step 7 — Testing

Monolithic Architecture and **Microservices Architecture** behave in different situations.

1. When the User Service/Module Fails

Monolithic Architecture:

If the User module has a serious failure, it may affect the entire application because all modules run together in one application.

Microservices Architecture:

If the User Service fails, the Product and Order Services can continue working if they do not require the User Service for that operation. The failure can be isolated to one service.

2. When the Database Fails

Monolithic Architecture:

Since all modules commonly use one database, a database failure can make the entire application unavailable.

Microservices Architecture:

Each service can have its own database. If one service database fails, only that particular service may be affected, while other services can continue working.

3. When the Order Service Receives Heavy Traffic

Monolithic Architecture:

The entire application may need to be scaled, including the User and Product modules, even though only orders have heavy traffic.

Microservices Architecture:

Only the Order Service can be scaled by adding more instances or resources. This makes better use of resources.

